The LQD Model

A log-quantile-density parametrization for arbitrage-free equity smiles

Vol-Fitter Technical Notes, No. 01

Vol-Fitter

July 26, 2026

Abstract

This note develops the slice and surface parametrization that the Vol-Fitter calls *LQD* — "log quantile density". For a single expiry let $X = \log(S_T/F_T)$ be the log-forward return under the T-forward measure, let $Q(u)$ be its quantile function and $q(u) = Q'(u)$ its quantile density. The model parametrizes

$$\ell(u) = \log q(u) = -\log u - \log(1-u) + (1-u)L + uR + \sum_{n=2}^{N} a_n P_n(1-2u), \qquad u \in (0,1),$$

where the P_n are Legendre polynomials. The shipped default $N = 6$ has seven parameters (the API supports $4 \leq N \leq 16$). Because $q = e^{\ell} > 0$ automatically, the recovered risk-neutral density is non-negative *by construction*: a one-expiry LQD slice is free of butterfly arbitrage after a single scalar martingale normalization, with no positivity penalty anywhere in the optimizer. The logarithmic endpoint terms give exponential return tails, hence linear implied-variance wings that sit automatically inside Lee's moment bounds; one explicit integrability condition $A_R < 1$ guarantees a finite forward. We give the full pricing equations, the tail and Lee-slope analysis, the exact analytic ATM level/skew/curvature handles, the trader-facing ATM-orthogonal chart with its market-package shape controls, and the calibration objective with its soft tail barrier and its three optimization charts — the default *logistic* chart maps the admissibility wall to infinity, so the optimizer roams an unconstrained space that covers exactly the admissible slices. A dedicated section derives the model's *analytic residual Jacobian* — a clean cancellation identity that removes the finite-difference quadrature rebuild, covers every production residual block except the prior terms, and is, to our knowledge, the original engineering contribution of this implementation. Every figure and table is generated fresh by the production calibrator on *synthetic approximation benchmarks* (market-fit evidence lives in the backtest reports): the seven-parameter fit to an SPX-like SVI-JW target reaches a maximum implied-volatility error of 0.23 volatility basis points, and a thirteen-parameter fit reproduces a bimodal "double-hat" event density.

Contents

1 Purpose and the modelling choice

Most smile models parametrize implied volatility, total variance, or option price *directly*. That is convenient to quote but awkward to keep arbitrage-free: non-negativity of the implied risk-neutral density is then a *global nonlinear* condition on the curve, policed during optimization by penalties that the fitter can violate on the way to a solution. The LQD model inverts the construction. It parametrizes a probability law first — through the quantile density of the terminal return — and reads option prices and implied volatilities off that law by integration afterwards. The design objective is a single sentence:

Heuristic

A calibrated slice should be a smooth, flexible smile, yet *every* admissible parameter vector should already correspond to a genuine probability law for the terminal forward return. "Fit a curve, then check it is a density" becomes "fit *within* the space of densities, then read off implied volatility."

Invariant

1. Every parameter vector prices from a genuine non-negative density: butterfly-freedom is structural, never a penalty (proposition 1).
2. The only hard admissibility condition is $A_R < 1$ (a finite forward), guarded by a smooth barrier before the wall — and, in the default logistic optimization chart, the wall is not even reachable: every unconstrained iterate is an admissible slice (section 7.2).
3. The martingale normalization is a single scalar shift: it changes neither the density's shape, its tails, nor its positivity.
4. The ATM handles $(\sigma_0, s_0, \kappa_0)$ are exact closed-form functionals of the slice — the same three coordinates the graph extrapolator of Note 14 propagates.

The modelling object is the log-forward return

$$X_T = \log\left(\frac{S_T}{F_T}\right), \tag{1}$$

with F_T the forward for expiry T. We suppress discounting and deterministic carry by working in normalized, undiscounted units; the normalized call at log-moneyness $k = \log(K/F_T)$ is

$$C_T(k) = \mathbb{E}^T\left[(e^{X_T} - e^k)^+\right], \qquad \mathbb{E}^T[e^{X_T}] = 1, \tag{2}$$

the second equality being the martingale (forward) condition. Unless a maturity is displayed, every formula below is for one expiry.

Time convention. Three time objects are distinct in production and must not be conflated: T is the *expiry date*; t_T is its calendar year fraction; and τ_T is the *active variance time* — calendar time dilated by scheduled events (Note 11), with $\tau_T = t_T$ whenever the event clock is off or the calendar is empty. Every quantity of the form $w = \sigma^2 \times \text{time}$, every vega, and every ATM-handle annualization in this note uses τ_T: the calibrator receives `prepared.tau`, not the calendar fraction.

To keep the classical formulas readable we write a bare T inside them; wherever T appears in a time role ($w = \sigma^2 T$, $\sqrt{T}$ in a vega, w_0/T in a handle), read τ_T.

Units and vocabulary. A dollar quote converts to the normalized units above by $C^{\text{norm}} = C^{\$}/(D_T F_T)$, with D_T the discount factor to expiry. Φ and φ are the standard normal CDF and PDF. A *volatility basis point* (vol bp) is 10^{-4} in volatility (0.01 vol point). The *(forward Black) delta* of a strike is $\Phi(d_+)$ — used here only as a moneyness label. The *haircut band* is the bid–ask band shrunk by a fixed vol amount (Note 07). A *risk reversal* (RR) is the call-minus-put IV difference at matched deltas and a *butterfly* (BF) the wing-average-minus-ATM IV at matched deltas; both appear only as prior operators (Note 13). *SVI* is the "stochastic volatility inspired" smile parametrization and *SVI-JW* its "jump-wings" trader re-coordinatization (appendix C).

The whole model in one line. Coefficients $\theta = (L, R, a_2, \ldots, a_N)$ define g; integrating e^g gives the quantile Q; one scalar shift μ makes e^Q a mean-one law; the asset-share integral of e^Q prices calls; the Black inversion of those prices is the smile:

$$\theta \;\to\; g \;\to\; Q \;\to\; \mu \;\to\; C(k) \;\to\; \sigma_{\text{imp}}(k).$$

If F_X is the law of X and $Q(u) = F_X^{-1}(u)$ for $0 < u < 1$, the quantile density is

$$q(u) = Q'(u) = \frac{1}{f_X(Q(u))}. \tag{3}$$

Modelling $\ell = \log q$ forces $q > 0$, hence Q strictly increasing, hence a *bona fide* non-negative density. That is the entire structural reason a one-expiry LQD slice carries no butterfly arbitrage. The chosen basis is

$$\boxed{\ell(u) = -\log u - \log(1-u) + (1-u)L + uR + \sum_{n=2}^{N} a_n P_n(1-2u)} \tag{4}$$

and it is deliberately *not* a generic polynomial basis. The two singular terms $-\log u$ and $-\log(1-u)$ manufacture logarithmic quantile tails — equivalently, exponential return tails and therefore the linear implied-total-variance wings that Lee's formula demands. The endpoint coefficients L, R scale those tails; the Legendre modes a_n shape the body. The shipped default $N = 6$ carries the seven parameters $(L, R, a_2, a_3, a_4, a_5, a_6)$; the API accepts $4 \le N \le 16$, and higher N adds body detail without ever touching the no-arbitrage mechanism.

In code the positivity is a single line: with the smooth part $g(u) = (1-u)L + uR + \sum_n a_n P_n(1-2u)$, the return density is $f_X(Q(u)) = 1/q(u) = u(1-u)\,e^{-g(u)}$, non-negative for *any* coefficients.

Listing 1: The LQD density is structurally non-negative — no constraint, no penalty (distilled from `models/lqd/quadrature.py`).

```
def density(u, g):            # g(u) = (1-u) L + u R + sum a_n P_n(1-2u)
    return u * (1 - u) * np.exp(-g)    # >= 0 for ANY g: no butterfly
```

Why this is the right trade. A direct-IV model spends optimizer effort *policing* arbitrage; the LQD model spends none, because the feasible set *is* the set of densities. The price is two scalar nonlinearities: a martingale normalization (one log of an integral, section 2.2) and a tail admissibility condition $A_R < 1$ (section 4). Both are cheap, and both are imposed once per residual evaluation rather than at every quote.

2 Static arbitrage and the quantile construction

2.1 Normalized prices and Breeden–Litzenberger

Write $Y = e^X = S_T/F_T$ and let $\widetilde{C}(y) = \mathbb{E}[(Y-y)^+]$ be the normalized call as a function of the normalized strike $y = e^k$. A static, butterfly-free slice requires $\widetilde{C}$ decreasing and convex in y with $\widetilde{C}(0) = 1$ and $\widetilde{C}(\infty) = 0$. When Y has a density,

$$\frac{\partial \widetilde{C}}{\partial y} = -(1 - F_Y(y)), \qquad \frac{\partial^2 \widetilde{C}}{\partial y^2} = f_Y(y) \geq 0, \tag{5}$$

which is the Breeden–Litzenberger identity: the second strike derivative of the call *is* the risk-neutral density. The quantile construction supplies such a density directly. Given a strictly increasing Q for X, the quantile of $Y = e^X$ is $Q_Y(u) = e^{Q(u)}$; if $\int_0^1 e^{Q(u)} \mathrm{d}u = 1$ then $\mathbb{E}[Y] = 1$. For $k \in \mathbb{R}$ let u_k solve $Q(u_k) = k$; then

$$C(k) = \int_{u_k}^1 (e^{Q(u)} - e^k) \mathrm{d}u, \tag{6}$$

$$P(k) = \int_0^{u_k} (e^k - e^{Q(u)}) \mathrm{d}u, \tag{7}$$

and subtracting gives put–call parity $C(k) - P(k) = 1 - e^k$.

Proposition 1 (A single LQD slice is butterfly-free)**.** *Suppose $q(u) = e^{\ell(u)} > 0$ on $(0,1)$ and the martingale integral $\int_0^1 e^{Q(u)} \mathrm{d}u$ is finite. After the scalar shift of Q that makes this integral equal to one, the prices (6) are generated by a probability law on $Y > 0$ with mean one; the slice is therefore free of butterfly arbitrage.*

Proof. Since $q > 0$, Q is strictly increasing and defines a law for X; $Y = e^X > 0$; the shift enforces $\mathbb{E}[Y] = 1$. The call is the expectation of $(Y - e^k)^+$ under this law, so monotonicity and convexity in strike follow from (5). No separate positivity inequality is ever checked. □

One cross-series remark. The application separately runs a dense-grid belly *butterfly certificate* on every displayed slice, whatever its model, and hard-blocks publish on failure (Note 02 — the penalty-fenced overlay families genuinely need it). For an LQD slice the certificate is a measured confirmation of proposition 1, not a constraint the optimizer ever feels.

2.2 The logit coordinate and stable integration

The endpoint singularities in (4) vanish under the logit change of variable

$$z = \log \frac{u}{1-u}, \qquad u = \Lambda(z) = \frac{1}{1 + e^{-z}}. \tag{8}$$

Writing the smooth part $g(u) = (1-u)L + uR + \sum_{n\geq 2} a_n P_n(1-2u)$, we have $q(u) = e^{g(u)}/(u(1-u))$ and, crucially,

$$\frac{\mathrm{d}Q}{\mathrm{d}z} = q(u)\, u(1-u) = e^{g(\Lambda(z))}. \tag{9}$$

All singularities are gone: the quantile is the integral of a smooth positive function on the line,

$$\overline{Q}(z) = \int_0^z e^{g(\Lambda(s))} \mathrm{d}s, \tag{10}$$

and a scalar shift μ enforces the martingale constraint,

$$Q(z) = \mu + \overline{Q}(z), \qquad \mu = -\log\left(\int_{-\infty}^{\infty} e^{\overline{Q}(z)}\, \Lambda(z)(1-\Lambda(z))\, \mathrm{d}z\right). \tag{11}$$

This is the *only* normalization; it changes neither q, the tails, nor positivity. Pricing is equally clean. Let z_k solve $Q(z_k) = k$, $u_k = \Lambda(z_k)$, and define the *upper asset-share integral*

$$A(z) = \int_z^{\infty} e^{Q(s)}\, \Lambda(s)(1-\Lambda(s))\, \mathrm{d}s. \tag{12}$$

Since $1 - u_k = \int_{z_k}^{\infty} \Lambda(s)(1-\Lambda(s))\mathrm{d}s$, the call price is

$$C(k) = A(z_k) - e^k(1-u_k). \tag{13}$$

A single quadrature grid yields $Q(z)$ and $A(z)$; every strike is then priced by interpolation. This pricing identity is also the seed of the analytic Jacobian of section 8, because the implicit dependence of z_k on the parameters will cancel exactly.

> **Example: the logistic base — the whole machine on one toy**
>
> Set every $a_n = 0$ and $L = R = \log s$. Then $g \equiv \log s$, so (9) gives $\mathrm{d}Q/\mathrm{d}z = s$: the quantile is exactly $Q(z) = \mu + sz$, i.e. $X = \mu + s\log(U/(1-U))$ is a *logistic* random variable with scale s and variance $\pi^2 s^2/3$. The endpoint scales are $A_L = A_R = s$, so the slice is admissible iff $s < 1$; the shift (11) evaluates to $\mu = -\log(\pi s/\sin(\pi s))$ in closed form; and the smile is a gentle symmetric smile with $w_0 \approx \pi^2 s^2/3$. This two-line model is precisely the production cold-start initializer of section 7: match s to the ATM variance, then let the optimizer bend $L \neq R$ (tail asymmetry) and the a_n (body).

3 Basis functions and exact coefficients

3.1 The Legendre body

With $x = 1 - 2u \in (-1, 1)$ the Legendre polynomials obey $\int_{-1}^1 P_m P_n \mathrm{d}x = \frac{2}{2n+1}\mathbf{1}_{m=n}$, hence

$$\int_0^1 P_m(1-2u)P_n(1-2u)\mathrm{d}u = \frac{1}{2n+1}\mathbf{1}_{m=n}. \tag{14}$$

This is a well-conditioned global basis for the smooth part of ℓ. The first two degrees are carried by the endpoint terms, since

$$(1-u)L + uR = \frac{L+R}{2} + \frac{L-R}{2}(1-2u), \tag{15}$$

so the seven-parameter $N = 6$ model spans the same polynomial space as $P_0, \dots, P_6$ while preserving the endpoint meaning of the first two coordinates. The body polynomials are built by the stable three-term recursion

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \qquad P_0 = 1,\ P_1 = x. \tag{16}$$

3.2 Endpoint scales

The endpoint values of g set the tail scales. Because $P_n(1) = 1$ and $P_n(-1) = (-1)^n$,

$$A_L := e^{g(0)} = \exp\Big(L + \textstyle\sum_{n\ge 2} a_n\Big), \tag{17}$$

$$A_R := e^{g(1)} = \exp\Big(R + \textstyle\sum_{n\ge 2} (-1)^n a_n\Big). \tag{18}$$

These are not diagnostics — they are the slopes of the quantile in the logit coordinate, $Q(z) = \mu + A_L z + O(1)$ as $z \to -\infty$ and $Q(z) = \mu + A_R z + O(1)$ as $z \to +\infty$. The right-tail integrability condition for a *finite forward* is

$$A_R < 1. \tag{19}$$

Indeed, as $z \to +\infty$, $\Lambda(z)(1-\Lambda(z)) \sim e^{-z}$ and $e^{Q(z)} \sim Ce^{A_R z}$, so the martingale integrand decays like $e^{(A_R-1)z}$, integrable iff $A_R < 1$. On the left the integrand decays like $e^{(A_L+1)z}$ for every $A_L > 0$, so no left condition is needed.

Caution

$A_R < 1$ is a genuine hard constraint, not a regularization preference: an LQD slice with $A_R \ge 1$ has an *infinite* forward and its "prices" are meaningless. The calibrator enforces it with a hard rejection *and* a smooth soft barrier that engages well before the boundary (section 7), so the finite-difference and analytic Jacobians stay smooth as the optimizer approaches the wall.

4 Tail asymptotics and Lee moment boundaries

4.1 Return moments

The endpoint linearity of Q in z gives exponential return tails. For large x, $z \sim x/A_R$ and $1-u \sim e^{-z}$, so

$$\mathbb{Q}(X > x) \asymp e^{-x/A_R}, \qquad x \to +\infty, \tag{20}$$

and symmetrically $\mathbb{Q}(X < x) \asymp e^{x/A_L}$ as $x \to -\infty$. The moment generating function of X therefore has critical strip $-1/A_L < r < 1/A_R$, and since $Y = e^X$ has mean one,

$$p_R^* = \sup\{p \ge 0 : \mathbb{E}[Y^{1+p}] < \infty\} = \frac{1}{A_R} - 1, \qquad q_L^* = \sup\{q \ge 0 : \mathbb{E}[Y^{-q}] < \infty\} = \frac{1}{A_L}. \tag{21}$$

4.2 Lee wing slopes

Let $w(k,T) = \sigma^2_{\mathrm{BS}}(k,T)\,T$ be total implied variance. Lee's moment formula gives the asymptotic total-variance slopes $\beta_R = \limsup_{k\to+\infty} w/k$ and $\beta_L = \limsup_{k\to-\infty} w/|k|$ as

$$\beta = \psi(p), \qquad \psi(p) = 2 - 4(\sqrt{p^2+p} - p), \qquad p \in [0,\infty], \tag{22}$$

equivalently $\frac{1}{2\beta} + \frac{\beta}{8} - \frac{1}{2} = p$, $0 < \beta \le 2$. Combining with (21),

$$\beta_L = \psi\left(\frac{1}{A_L}\right), \qquad \beta_R = \psi\left(\frac{1}{A_R} - 1\right), \quad A_R < 1. \tag{23}$$

The endpoint coefficients thus do double duty: they regularize extrapolation *and* identify the moment critical exponents and the Lee-consistent wing slopes. A calibration may fit L, R freely (subject only to $A_R < 1$) or impose explicit wing priors by constraining A_L, A_R through (17)–(18). The cross-model treatment of wings and Lee bounds is the subject of Note 09; here they fall out of the parametrization for free.

Heuristic

The map ψ is decreasing: a *heavier* tail (larger A, hence smaller critical exponent) gives a *steeper* variance wing (larger β). Be precise about "fat": LQD return tails are *exponential*, hence asymptotically *much* heavier than the log-normal's Gaussian log-return tails — though over traded strikes the two can look close. Equity index smiles fit small A_L, A_R, so their wings sit comfortably below the model-free ceiling $\beta = 2$, and a truncation $z \in [-40, 40]$ in double precision is far more than enough (section 5.2).

5 Option pricing and implied-volatility recovery

5.1 The Black map and vega-normalized residuals

The normalized Black call in total-variance units is

$$B(k,w) = \Phi(d_+) - e^k\Phi(d_-), \qquad d_\pm = -\frac{k}{\sqrt{w}} \pm \frac{\sqrt{w}}{2}, \tag{24}$$

and the implied total variance generated by the slice solves $B(k, w(k,T)) = C_T(k)$. For calibration the loss is expressed in vega-scaled price units rather than raw price, so that it approximates a volatility error while preserving the exact arbitrage-free price construction during the solve: given quoted volatility σ_i at k_i, with $w_i = \sigma_i^2 T$,

$$r_i(\theta) = \frac{C^{\mathrm{LQD}}(k_i;\theta) - B(k_i,w_i)}{\partial_\sigma B(k_i,w_i) + \eta}, \qquad \partial_\sigma B = \varphi(d_+)\sqrt{T}, \tag{25}$$

with a small floor η in the far wings (appendix A, $\eta =$ `_VEGA_FLOOR`). Working in price space keeps every iterate a valid density; the vega division undoes the price-to-vol scale so the optimizer sees something close to a vol residual.

5.2 Quadrature, Hermite interpolation, and Newton inversion

On a finite logit grid $z_j \in [-Z, Z]$ the pipeline computes, in one pass, $u_j = \Lambda(z_j)$, $h_j = e^{g(u_j)}$, the cumulative $\overline{Q}_j$ by (fourth-order) cumulative Simpson quadrature, the shift μ from (11), and the reverse-cumulative asset share A_j. The two truncation tails are added analytically: at the right,

$$\int_Z^\infty e^Q \Lambda(1-\Lambda)\mathrm{d}z \approx \frac{e^{Q(Z)-Z}}{1-A_R}, \qquad \int_{-\infty}^{-Z} e^Q \Lambda(1-\Lambda)\mathrm{d}z \approx \frac{e^{Q(-Z)-Z}}{1+A_L}. \tag{26}$$

For equity tails with A_L, A_R small, $Z = 40$ buries the truncation error far below double-precision vega noise.

Two implementation refinements matter for accuracy and for the analytic Jacobian. First, the production build stores the *exact nodal derivatives* $\mathrm{d}Q/\mathrm{d}z = e^g$ and $\mathrm{d}A/\mathrm{d}z = -e^Q u(1-u)$ alongside the nodal values, so both Q and A are interpolated off-grid with *cubic Hermite* polynomials, not linear interpolation. Priced curves are then smooth enough for clean finite-difference Greeks, and — because Hermite interpolation returns the nodal value exactly at a node — calendar constraints expressed at grid z-values match node-indexed ones bit-for-bit. Second, the strike inversion $Q(z_k) = k$ is solved by *Hermite–Newton* iteration to machine precision, not by a coarse table lookup, which keeps the ATM handles (section 6) exact.

Performance

The quadrature grid $(z, u, u(1-u))$ and the Legendre matrix $P_n(1-2u)$ depend only on (Z, n_{pts}) and the model order — never on θ. A single calibration builds the slice many hundreds of times, so these arrays are cached (returned read-only) and only the parameter-dependent combination $g(u)$ is re-formed per call; this alone made `build_slice` about 1.7× faster. Optimization runs on a coarser $n_{\mathrm{pts}} = 2001$ grid whose Simpson error is already orders of magnitude below the fit budget, and the *accepted* slice is rebuilt once at the full grid for reporting, density, var-swap and calendar diagnostics. See appendix B.

6 Exact ATM handles and the orthogonal chart

6.1 Level, skew and curvature from the quantile

Traders want the first coordinates of a model to be *actual* ATM level, skew and curvature. The LQD slice delivers these in closed form, with no finite differencing of implied volatility. Let u_0 solve $Q(u_0) = 0$, set $c_0 = C(0)$, and let $f_0 = f_X(0) = u_0(1-u_0)e^{-g(u_0)}$ be the ATM return density. The first two *log-strike* call derivatives are exactly

$$C'(0) = -(1-u_0), \qquad C''(0) = f_0 - (1-u_0), \tag{27}$$

where C'' is in log-strike (from $C'(k) = -e^k(1-F_X(k))$). Let w_0 solve the ATM Black identity $B(0, w_0) = 2\Phi(\sqrt{w_0}/2) - 1 = c_0$. With $a = \sqrt{w_0}/2$, $n = \varphi(a)$, $\Phi_a = \Phi(a)$, the Black partials at $k = 0$ are

$$\begin{aligned} B_k &= -(1-\Phi_a), \quad & B_w &= \tfrac{n}{2\sqrt{w_0}}, \quad & B_{kk} &= -(1-\Phi_a) + \tfrac{n}{\sqrt{w_0}}, \\ B_{kw} &= \tfrac{n}{4\sqrt{w_0}}, \quad & B_{ww} &= -\tfrac{n}{4w_0^{3/2}} - \tfrac{n}{16\sqrt{w_0}}. \end{aligned} \tag{28}$$

Differentiating $B(k, w(k)) = C(k)$ once and twice at $k = 0$ gives

$$w_1 = \frac{C'(0) - B_k}{B_w}, \qquad w_2 = \frac{C''(0) - B_{kk} - 2B_{kw}w_1 - B_{ww}w_1^2}{B_w}, \tag{29}$$

and the actual implied-volatility handles are

$$\sigma_0 = \sqrt{\tfrac{w_0}{T}}, \quad s_0 = \frac{w_1}{2\sqrt{Tw_0}}, \quad \kappa_0 = \frac{w_2}{2\sqrt{Tw_0}} - \frac{w_1^2}{4\sqrt{T}\,w_0^{3/2}}. \tag{30}$$

These are exact for every LQD slice, and their meaning and units are worth stating: $\sigma_0 = \sigma_{\text{imp}}(0)$ is the ATM implied volatility (annualized on the active clock τ_T), $s_0 = \partial_k\sigma_{\text{imp}}(0)$ is the ATM *skew* (volatility per unit log-moneyness) and $\kappa_0 = \partial_{kk}\sigma_{\text{imp}}(0)$ the ATM *curvature* (volatility per unit log-moneyness squared). Two closely related coordinate triples coexist in production and must not be conflated: the *graph-facing carrier* is $h = (\sigma_0, s_0, \kappa_0)$ — what Note 14 propagates across the smile graph — while the *internal orthogonal chart* of section 6.2 works in $H = (w_0, s_0, \kappa_0)$ with $w_0 = \sigma_0^2\,\tau_T$ the ATM total variance (`models/lqd/ortho.py`; the graph adapter converts between the two).

6.2 The ATM-orthogonal chart

Collect the three handles into $H(\theta) = (w_0, s_0, \kappa_0)$ — note the first coordinate is the ATM *total variance*, not σ_0 (section 6.1) — and let $J = \partial H/\partial\theta \in \mathbb{R}^{3\times d}$ at a reference θ^* (computed by central differences; the quadrature is deterministic, so this is accurate to $\sim 10^{-9}$). The least-norm perturbation realizing a desired handle change δh is $\delta\theta = J^\top(JJ^\top)^{-1}\delta h$, and $\Pi_\perp = I - J^\top(JJ^\top)^{-1}J$ projects onto handle-preserving directions. With $U = J^\top(JJ^\top)^{-1}$ (so $JU = I_3$) and V an orthonormal basis of $\ker J$ from the QR factorization of $\Pi_\perp$, the chart

$$\theta = \theta^* + U\alpha + V\xi \tag{31}$$

has *three primary coordinates* α that move the trader handles and *shape coordinates* ξ that leave them fixed to first order. For larger moves the map is made exact by a three-dimensional Newton solve of $H(\theta^* + U\alpha + V\xi) = h^{\text{target}}$ (the "retarget"); because $JU = I_3$ the 3×3 Jacobian is the identity at the reference, an excellent preconditioner that rarely needs refreshing. This is what lets a trader drag ATM level, skew or curvature while the optimizer keeps using the remaining Legendre modes for wings, shoulders and event convexity.

Caution

The chart is *local*: U and V are built from the Jacobian at the reference θ^*, so the first-order handle-invariance of the shape directions degrades for large excursions, and the retarget Newton solve can fail to converge — or converge only to an *infeasible* point — when the requested handles push the slice toward the $A_R = 1$ integrability wall (e.g. a large curvature move on a thin-tailed reference). Callers must treat retargeting as an operation that can fail near the wall, not as a total map on handle space.

6.3 Shape controls in market vocabulary

The chart's kernel basis V is unique only up to rotation: "shape direction 1" can rotate or flip between calibrations — a sound interface for one-session sculpting, not a persistent trader control. The stable vocabulary is the market's own quote packages. For the package vector $P(\theta) = (\mathrm{RR25}, \mathrm{BF25}, \mathrm{RR10}, \mathrm{BF10}, \mathrm{VarSwap})$, evaluated on the slice's own smile, restrict its Jacobian to $\ker J$ and take the least-norm kernel combination that moves *one* package by one unit while the handles stay put to first order,

$$\xi_i = \arg\min_{\xi} \ |\xi| \quad \text{s.t.} \quad \left(\tfrac{\partial P}{\partial \theta} V\right)\xi = e_i, \tag{32}$$

so $V\xi_i$ is "one unit of package i, handles fixed, minimal shape motion" (`models/lqd/packages.py`). The accompanying *cross-talk matrix* prints how much each package direction moves the *other* packages — the honest statement of how independent the controls really are on a given slice: they are exactly independent only to first order and only at full kernel rank, and the rank and SVD condition are reported rather than assumed. Orthogonal to this, the chart's notion of "least norm" is itself a choice: passing the Gauss–Newton (Fisher) metric $G = J_r^{\mathsf{T}} J_r$ of the converged fit re-prices moves by *fit impact* — the cheapest handle move becomes the one that disturbs the fitted quotes least, which is the economically defensible reading. The Euclidean metric remains the default (byte-identical to the historical chart); promoting the GN metric to default is an open decision, not a shipped one.

7 Calibration of one expiry

7.1 The objective

Given quotes $(k_i, \sigma_i, \omega_i)$ at one maturity, with $w_i = T\sigma_i^2$, the calibrator minimizes

$$\min_{\theta} \ \sum_i \omega_i \left(\frac{C^{\mathrm{LQD}}(k_i;\theta) - B(k_i, w_i)}{\partial_\sigma B(k_i, w_i) + \eta} \right)^2 + \lambda \sum_{n \geq 4} n^{2r} a_n^2, \tag{33}$$

as a bound-free nonlinear least-squares problem (scipy `trf`, in the active optimization chart of section 7.2). The high-order ridge $\lambda\, n^{2r} a_n^2$ is optional and suppresses oscillation in sparse markets; it deliberately leaves the first body modes a_2, a_3 free. The hard admissibility condition is $A_R < 1 - \varepsilon_R$ for a small buffer ε_R. There is *no* density-positivity term — positivity is structural (proposition 1). Equation (33) shows only the *core* rows (data + ridge); the full production residual stack, with each block's activation condition and Jacobian route, is tabulated in section 7.4.

The soft tail barrier. To keep the residual smooth as the optimizer approaches the $A_R = 1$ wall, the stacked residual carries a softplus barrier

$$b(\theta) = \log\bigl(1 + e^{\,s(A_R - c)}\bigr), \tag{34}$$

with centre $c = 0.90$ and steepness $s = 50.0$ (both `FitSettings` coefficients; evaluated as $\mathrm{logaddexp}(0,\cdot)$ so a wild trial θ cannot overflow it). The barrier is negligible for $A_R \ll c$ and rises steeply near the wall, pushing the fit back *before* it reaches the hard rejection. When a trial θ does cross $A_R \geq 1$, the price block is replaced by a large smooth-in-A_R penalty $10 + A_R$ so the trust region still has a gradient pointing back to feasibility.

7.2 The optimization charts

The minimization variable need not be θ itself. Production offers three *charts* over the same model family (`lqdCoords`; `models/lqd/charts.py`). A chart changes the optimizer's trust-region geometry, never the objective, so the fitted optimum is chart-independent to solver tolerance and the canonical storage and wire format stay $\theta = (L, R, a_2, \ldots, a_N)$ everywhere:

- `lr` — the historical identity chart: optimize θ itself.
- `endpoint` — $\phi = (\log A_L, \log A_R, a_2, \ldots, a_N)$, an exact unit-determinant *linear* map: holding the first two coordinates fixed while moving a body mode automatically counter-adjusts (L, R) to keep both endpoint scales, so the coefficients that fit acute central convexity can no longer mechanically drag the asymptotic wings while the solver moves.
- `logistic` (the default) — the endpoint chart with the right tail pushed through the logistic, $A_R = \mathrm{expit}(\rho)$. The wall $A_R = 1$ maps to $\rho = +\infty$: *every* finite iterate is an admissible slice, so the chart covers exactly the admissible set and the least-squares problem is genuinely unconstrained. Near the wall the chart compresses ($\mathrm{d}\log A_R/\mathrm{d}\rho = 1 - A_R \to 0$), damping trust-region steps precisely where the raw charts need the soft barrier; far below the wall $\rho \approx \log A_R$ and the two charts agree to first order.

The analytic Jacobian of section 8 rides along by the chain rule (the chart maps are linear, plus one diagonal logistic factor). One numerical footnote keeps the claim honest: the logistic removes the *mathematical* wall, not the floating-point one — beyond $\rho \approx 36$, A_R rounds to exactly 1.0 in double precision, so the `EPS_AR` buffer remains the hard numerical guard and the barrier (34), part of the objective in every chart, keeps the optimizer clear of that region.

7.3 Initialization

The cold-start initializer is the *logistic base*: $a_n = 0$, $L = R = \log s$ with $s = \sqrt{3w_0}/\pi$ from the variance match $\mathrm{Var}(X) = \pi^2 s^2/3$, seeded from the ATM-interpolated quote variance — the `logistic_init` routine of `calibrate.py`, and exactly the toy of the example in section 2.2. It is the only cold initializer in production. Sequential workflows do better than cold: the background Calibrate sweep *warm-starts* each expiry from the previous, shorter-dated expiry's freshly fitted parameters (adjacent maturities have nearly the same smile, so `trf` lands close to the optimum), and the service can also run a *data-only prepass* — a persistence-free fit used as the measurement stage of the prior and filter machinery (Notes 13, 15). Wing-aware and quantile-projection initializers are roadmap items, not shipped code.

7.4 The full residual stack

The production objective is the stacked residual of table 1 — `calibrate_slice` concatenates, in order: the data block, the ridge, the calendar slack, the barrier, and the optional var-swap/prior blocks. Every optional block is byte-identical-to-absent when unused (test-locked), and each connects this note to the rest of the series. The Jacobian column matters operationally: the analytic route of section 8 covers every block except the prior-anchor and operator rows — the market and prior var-swap rows ride the same sensitivity pass since committee revision R5 — so only a prior-anchor or operator block switches the whole fit to scipy's finite-difference Jacobian (correct, merely not accelerated).

Residual block	Active when	Units / scaling	Jacobian
Mid data block	`fitMode=mid` (default)	vega-normalized price ($\approx$ vol)	analytic
Band data block (hinge + mid anchor; Note 07)	`fitMode` `bidask/haircut`	vega-normalized price	analytic
High-order ridge	`regLambda`> 0 (rows $n \geq 4$)	coefficient units	analytic
Calendar slack (Note 10)	`enforceCalendar` + a nearer fitted expiry	normalized price at fixed strike, support-confined and tapered (section 9)	analytic
A_R soft barrier	always (one row)	dimensionless	analytic
Market var-swap (Note 08)	`varSwapEnabled` + an active quote	vol	analytic (R5)
Prior strike-gap anchor (Note 13)	`strike_gap` / `hybrid` tail, active prior	vega-normalized price	FD fallback
Prior var-swap companion (Note 13)	alongside an active prior	vol	analytic (R5)
Operator prior (Note 13)	`quote_operator` / `hybrid`, gated per operator	operator units (vol)	FD fallback
Active-filter MAP (Note 15)	filter mode `active` (rides the operator block, ungated)	handle units	FD fallback

Table 1: The LQD residual stack as production assembles it (`models/lqd/calibrate.py`; the prior/filter targets are resolved in `api/service.py`). "FD fallback" means the presence of that block switches the entire fit to the finite-difference Jacobian.

7.5 The var-swap route: a native log-contract quadrature

The log-contract identity $w_{\mathrm{vs}} = -2\,\mathbb{E}[X] = -2\int_0^1 Q(u)\mathrm{d}u$ is exact for the log-contract *proxy* of the var swap; LQD evaluates it as a single trapezoidal quadrature on the grid it already owns,

$$w_{\mathrm{vs}} = -2\int_{-\infty}^{\infty} Q(z)\,\Lambda(z)(1-\Lambda(z))\,\mathrm{d}z, \tag{35}$$

whose integrand decays like $z\,e^{-|z|}$ (`quadrature.py::var_swap_strike`). Two qualifications keep the claim honest. First, this is a *direct native one-dimensional quadrature*, dramatically cheaper than the generic strike replication of Note 08 — which would re-solve implied variance on a strike grid at every Jacobian column and turn a millisecond fit into minutes. The row also stays on the analytic Jacobian path (committee revision R5): $\partial w_{\mathrm{vs}}/\partial\theta = -2\int(\partial Q/\partial\theta)\,\Lambda(1-\Lambda)\,\mathrm{d}z$ rides the nodal quantile sensitivities the pass of section 8 already owns, so the historical gate that sent every var-swap-enabled fit to finite differences is lifted. Second, the identity prices the log contract: its correspondence with a realized-variance swap assumes the continuous-monitoring, continuous-path (diffusion) convention; jumps and discrete sampling require the usual corrections, which are outside this model's scope.

8 The analytic residual Jacobian

The dominant cost of (33) used to be the Jacobian: scipy rebuilt the entire quadrature pipeline $P+1$ times per iteration to finite difference it ($P = d = N+1$ parameters). The Vol-Fitter instead propagates the analytic Jacobian of the residual stack — the mid or band data block, ridge, calendar slack, barrier and var-swap rows of table 1 — in *one* quadrature pass; only a fit with a prior-anchor or operator block active falls back to finite differences. The key is a cancellation identity, and it is the cleanest piece of mathematics in the implementation.

8.1 The cancellation identity

Regard the priced call (13) as a function of θ through both the slice and the strike root $z_k(\theta)$ that solves $Q(z_k;\theta) = k$:

$$C(k;\theta) = A(z_k;\theta) - e^k(1 - \Lambda(z_k)). \tag{36}$$

Differentiating,

$$\frac{\mathrm{d}C}{\mathrm{d}\theta} = \underbrace{\frac{\partial A}{\partial \theta}\Big|_z}_{\text{explicit}} + \Big[\underbrace{A_z(z_k)}_{=-e^{Q(z_k)}\Lambda(1-\Lambda)} + e^k\Lambda'(z_k)\Big]\frac{\mathrm{d}z_k}{\mathrm{d}\theta}. \tag{37}$$

Now $A_z(z) = -e^{Q(z)}\Lambda(z)(1-\Lambda(z))$ from (12), and at z_k we have $e^{Q(z_k)} = e^k$ and $\Lambda'(z_k) = \Lambda(1-\Lambda) = u_k(1-u_k)$. Hence the bracket is

$$A_z(z_k) + e^k\Lambda'(z_k) = -e^k u_k(1-u_k) + e^k u_k(1-u_k) = 0. \tag{38}$$

Theorem 1 (Implicit-root cancellation)**.** *The implicit dependence of the strike root z_k on the parameters drops out of the price gradient, leaving*

$$\frac{\mathrm{d}C}{\mathrm{d}\theta} = \frac{\partial A}{\partial \theta}\Big|_{z=z_k}. \tag{39}$$

So $\mathrm{d}C/\mathrm{d}\theta$ is obtained by evaluating the *nodal* parameter-sensitivities of the asset share at the fixed point z_k — by the very same Hermite interpolation used to price, fed the nodal sensitivities $\partial A/\partial\theta_j$ and $\partial A_z/\partial\theta_j$ instead of the nodal values.

One honesty note on exactness. Theorem 1 is exact for the *continuous* objects. The production discretization interpolates Q and A as two separate Hermite interpolants, so between grid nodes the interpolated price is not *algebraically* the integral of the interpolated quantile, and the implemented gradient inherits that (tiny) discretization mismatch. The operational claim is therefore *benchmarked numerical agreement*, not algebraic identity: the analytic Jacobian matches a 3-point finite difference to a relative 10^{-3} tolerance on the mid, band and calendar stacks, and analytic and finite-difference *fits* reach the same solution ($|\Delta\theta| \lesssim 10^{-5}$, cost agreement $\sim 10^{-11}$; `test_lqd_jacobian.py`).

8.2 Nodal sensitivities

Everything reduces to differentiating (10)–(12) term by term, using that g is *affine* in θ with constant basis $\phi_j = \partial g/\partial\theta_j$ — namely $\phi_L = 1-u$, $\phi_R = u$ and $\phi_{a_n} = P_n(1-2u)$. Since $\overline{Q}' = e^g$,

$$\frac{\partial \overline{Q}(z)}{\partial \theta_j} = \int_0^z e^{g(\Lambda(s))}\phi_j(\Lambda(s))\,\mathrm{d}s, \tag{40}$$

again by cumulative quadrature, anchored at the centre node. The martingale shift $\mu = -\log M$ with $M = \int e^{\overline{Q}}\Lambda(1-\Lambda)\mathrm{d}z$ differentiates to

$$\frac{\partial \mu}{\partial \theta_j} = -\frac{1}{M}\int e^{\overline{Q}}\,\frac{\partial \overline{Q}}{\partial \theta_j}\,\Lambda(1-\Lambda)\,\mathrm{d}z \;+\; \text{(the two analytic tail-correction terms)}, \tag{41}$$

where the tail corrections (26) contribute their own θ-derivatives through $\overline{Q}(\pm Z)$ and through A_L, A_R (whose gradients are $A_L\,(\mathbf{1},0,\mathbf{1}\ldots)$ and $A_R\,(0,\mathbf{1},(-1)^n\ldots)$ from (17)–(18)). Then $\partial Q/\partial\theta_j = \partial\mu/\partial\theta_j + \partial\overline{Q}/\partial\theta_j$, the asset-share sensitivity follows by reverse cumulative quadrature of $\partial(e^{Q}u(1-u))/\partial\theta_j$ plus its right-tail correction, and the nodal $\partial A_z/\partial\theta_j$ is just $-\partial(e^{Q}u(1-u))/\partial\theta_j$. The remaining residual blocks differentiate trivially: the band hinge multiplies $\mathrm{d}C/\mathrm{d}\theta$ by the violation sign, the ridge block is diag(reg) on the a-columns, the confined calendar rows (43) are $\sqrt{w_{\mathrm{cal}}}\,\mathrm{taper}_m\,(-\mathrm{d}C/\mathrm{d}\theta)$ at the active strikes (the legacy quantile form differentiates as $-\mathrm{d}A/\mathrm{d}\theta$ at its active z-nodes), the var-swap rows ride $\partial w_{\mathrm{vs}}/\partial\theta = -2\int(\partial Q/\partial\theta)\,\Lambda(1-\Lambda)\,\mathrm{d}z$, and the barrier row is $\mathrm{expit}\big(s(A_R-c)\big)\,s\,\partial A_R/\partial\theta$.

8.3 Scope and measured speed-up

The analytic Jacobian covers every configuration without prior-anchor or operator rows; committee revision R5 lifted the historical var-swap gate (the market and prior var-swap rows ride the same sensitivity pass), leaving only the prior terms on scipy's two-point finite difference — correct, merely not accelerated. Table 2 reports a fresh comparison: the analytic and finite-difference fits reach the same fitted solution within the stated tolerances (costs agreeing to ~ 11 significant figures, relative difference $\sim 10^{-11}$) while the analytic Jacobian is 1.3–1.7× faster across $N = 6\ldots12$ on this benchmark, with both paths converging in a handful of iterations. Scope of the measurement: the generator times the least-squares *residual path* (repeated `calibrate_slice` calls on the synthetic benchmark), not the full public calibration-and-reporting workflow; it runs single-threaded on this development machine with the static-grid cache warm and only the core residual blocks active. Different hardware, cache state or an active var-swap/prior block will move the ratio.

Table 2: Analytic vs. two-point finite-difference Jacobian, fresh run of the production calibrator on the SVI-JW benchmark (fastest of three repeats, single-threaded, warm static-grid cache, core residual blocks only). $P = N + 1$ is the parameter count; the cost column confirms the same optimum within tolerance.

N	P	analytic (ms)	FD (ms)	speed-up	$\lvert\Delta\text{cost}\rvert$
6	7	15.5	24.7	1.59×	5.6e-20
8	9	18.4	24.9	1.35×	1.4e-19
10	11	25.6	68.8	2.68×	8.9e-20
12	13	30.7	47.6	1.55×	2.0e-20

Heuristic

Why only $\sim 1.5\times$ and not $\sim P\times$? Two reasons. First, the cached static grid makes a *repeat* `build_slice` cheap, so the $P + 1$ finite-difference builds are not $P + 1$ times the cost of one cold build. Second, `trf` converges here in ~ 7 iterations, so the linear algebra and the single value evaluation dilute the Jacobian saving. The win is larger on heavier configurations, and the benchmark reaches the same fitted solution within the stated tolerances. The structural payoff — a Jacobian that no longer rebuilds the quadrature — is what makes higher-order and surface-wide fits comfortable.

9 Soft calendar-arbitrage control

Proposition 1 kills butterfly arbitrage within a slice but says nothing across expiries. Two standing assumptions frame this section: carry is deterministic (so working under each T-forward measure in normalized units is legitimate) and every slice is normalized to mean one, $\mathbb{E}[Y_T] = 1$. For $T_1 < \cdots < T_M$, absence of calendar arbitrage at fixed log-moneyness is $C_{T_i}(k) \le C_{T_j}(k)$ for $T_i < T_j$ — equivalently the laws of Y_T increase in *convex order*. In plain language: the longer-dated law is "more spread out" — every convex payoff (every call, at every strike) is worth at least as much under it — without its mean moving. With equal means, $Y_{T_i} \le_{cx} Y_{T_j}$ iff the *integrated upper-quantile* curves are ordered,

$$G_i(\alpha) := \int_\alpha^1 e^{Q_i(u)} \mathrm{d}u \ \le\ G_j(\alpha) \quad \forall \alpha \in [0,1], \tag{42}$$

with equality at $\alpha = 0$. This is exactly the upper asset share of (12) read at quantile level, so it is already on the grid — and that convenience turned out to be a trap worth documenting.

Case file: the acute weekly that bulldozed the ladder

Setup. An acutely convex short-dated slice (an event week) at the front of an otherwise consistent ladder, surface coupling on — the certification reproduction. **Failure mode.** Every longer-dated fit came off its quotes: the far expiry fitted 10.6 vol bp free and 1095 vol bp under the coupling. **Diagnosis.** The floor was sampled in G-space on the full grid. $G(\alpha)$ integrates the *whole upper tail*, so a comparison at any α leaks the near slice's extrapolated wing into the constraint — the confinement principle of Note 09 violated in quantile coordinates. Nor is a z-windowed G floor a fix: it removes only about half the drag, because the integral inside the window still carries the tail. **Fix.** Enforcement moved to *price space at fixed strike*, confined to the common quote support (below). **Verdict.** The far fit returned to 10.6 vol bp with the identified violation still repaired; the certification case `symmetric_calendar` locks the reproduction.

Production therefore enforces the ordering as *support-confined price rows*: on a tapered strike grid k_m spanning the two slices' common quote support, the later slice carries hinge rows

$$\sqrt{w_{\text{cal}}}\ \text{taper}_m \left(C_{\text{near}}(k_m) - C_{\text{far}}(k_m)\right)^+, \tag{43}$$

with $w_{\text{cal}} =$ `calendarWeight` (`calib/calendar.py::confined_calendar_floor`); the taper fades the active set smoothly as quoted spans shift between refits, and the rows sit on the analytic-Jacobian path like every other core block. The full-grid quantile floor survives as a legacy threading form and a diagnostic, not as the production constraint.

Two orchestrations consume these rows (`surfaceSolver`). The *sequential* build — nearest-to-farthest, each new slice fitted against its predecessor's floor — is the historical construction, kept as an explicit option; its defect is traversal-order bias: the first slice is immutable and every later slice absorbs its errors. The production default is the *symmetric* solver (`calib/symmetric.py`): fit every expiry *independently*; *screen* each adjacent interface for an *identified* violation — normalized-call ordering on the common quote support, vega-normalized so the number reads as a vol gap; then jointly Gauss–Newton-repair only the *violation-connected components*, stacking each member slice's own standalone residual blocks plus the tapered interface rows (43). Because every slice keeps its own quote weights, the correction is allocated by information: a liquid slice barely moves, an unsupported acute tail absorbs it. A clean ladder is *exactly* its independent fits, and if an interface stays violated after weight escalation the remainder is *irreducible slack* — genuinely inconsistent inputs — reported per interface, never silently flattened into the rest of the ladder.

Honesty about the guarantee: this is *control*, not a theorem. The hinge carries a finite weight, so a hard-pressed fit can retain a small violation rather than destroy the quote fit; the whole coupling is a user toggle (`enforceCalendar`) and off means off; and residual calendar violations are *measured and reported* rather than assumed away — since committee revision R5 in desk units too: the quality report prints the worst identified violation together with the strike where selling the near call against the far one pays most, converted to currency per contract, ticks and fraction of the quoted spread (`calendar_violation_argmax`). Note 10 develops the model-agnostic calendar machinery; here we only record that for LQD the natively *measured* object is the asset share $A(z)$, while the *enforced* object is the confined price row (43).

10 Worked examples

All numbers and figures in this section are produced by `Docs/notes/figures/gen_lqd.py`, which fits the production calibrator to two synthetic targets. Read them for what they are: *synthetic approximation benchmarks* — they measure how well the basis can reproduce a known clean curve, not how well the model fits noisy market quotes. Market-fit evidence (RMS vs bid–ask NBBO quotes, in- and out-of-sample, against an SVI-JW baseline) lives in the backtest harness and its reports (`backend/backtest/`).

10.1 An SPX-like SVI-JW smile

The benchmark target is a six-month SPX-like slice from raw SVI $w(k) = a + b\{\rho(k-m) + \sqrt{(k-m)^2 + \sigma_{\mathrm{SVI}}^2}\}$ with

$$(a,\, b,\, \rho,\, m,\, \sigma_{\mathrm{SVI}}) = (0.010625,\ 0.07289,\ -0.5,\ 0.05831,\ 0.10100),$$

an ATM volatility near 20.6% with a pronounced index put skew. The seven-parameter $N = 6$ fit over $k \in [-0.35, 0.30]$ converges in 7 residual evaluations to the coefficients of table 3, with endpoint and Lee diagnostics

$$A_L = 0.214458, \quad A_R = 0.069023, \quad \mu = 0.020613, \quad \beta_L = 0.097073, \quad \beta_R = 0.035757. \tag{44}$$

For comparison the raw-SVI wing slopes are $b(1-\rho) = 0.1093$ (left) and $b(1+\rho) = 0.0364$ (right): the Lee slopes agree to $\sim$2 digits on the right wing ($\beta_R = 0.035757$ vs 0.0364) and to $\sim$1 on the steeper left ($\beta_L = 0.097073$ vs 0.1093, $\sim$10%), even though nothing imposed it — they emerge from the fitted endpoint basis. The exact ATM handles are $\sigma_0 = 20.62\%$, $s_0 = -0.3538$, $\kappa_0 = 1.6474$, and the martingale check returns $\mathbb{E}[e^X] - 1 = 0.00e+00$. The maximum implied-volatility error over the calibration grid is 0.23 volatility basis points (fig. 1) — an approximation-capacity number on a clean synthetic target, not a market-fit error.

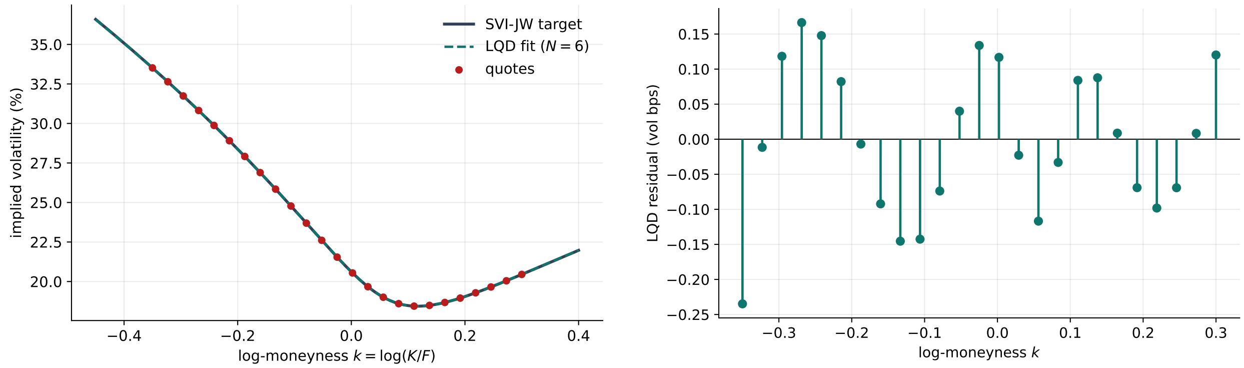

Figure 1: Left: the SVI-JW target, the seven-parameter LQD fit, and the quotes. Right: the LQD implied-volatility residual at the quotes, in volatility basis points. Beyond the calibration window the LQD curve is its own Lee-consistent extrapolation, not an SVI continuation.

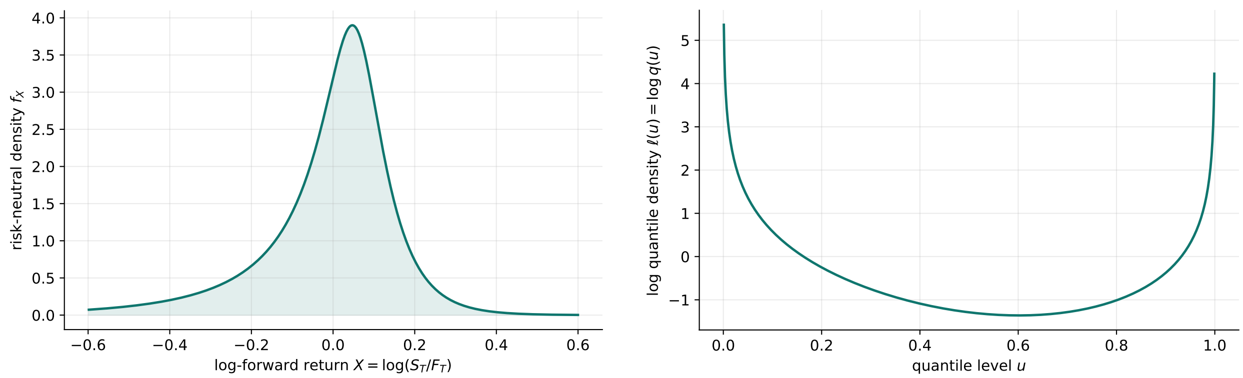

Figure 2: Left: the risk-neutral log-return density of the LQD slice fitted to the SVI target — non-negative by construction. Right: the fitted log quantile density $\ell(u) = \log q(u)$; the $-\log u - \log(1-u)$ endpoints are the source of the exponential tails.

Table 3: Seven-parameter LQD fit to the SPX-like SVI-JW slice (fresh run).

Parameter	Value
L	-1.87479110
R	-3.08957252
a_2	+0.38076853
a_3	-0.04700855
a_4	-0.00719617
a_5	+0.00645578
a_6	+0.00213226

10.2 A bimodal "double-hat" event smile

Ahead of a scheduled binary event the terminal density can be *bimodal*, a shape a low-order direct-IV model fights. The target here is an equal mixture of two log-return normals with $\sigma = 5\%$ centred near ±9–10% over $T = 30/365$, martingale-normalized. A thirteen-parameter $N = 12$ LQD fit over $k \in [-0.25, 0.25]$ recovers both the smile and the two-humped density (fig. 3), with endpoint scales $A_L = 0.015341$, $A_R = 0.014979$ and a maximum implied-volatility error of 13.19 volatility basis points. Positivity of the density is automatic even where the higher Legendre modes sculpt the two hats; in a sparse market one would lean on the ridge or the ATM-orthogonal chart to avoid spurious oscillation.

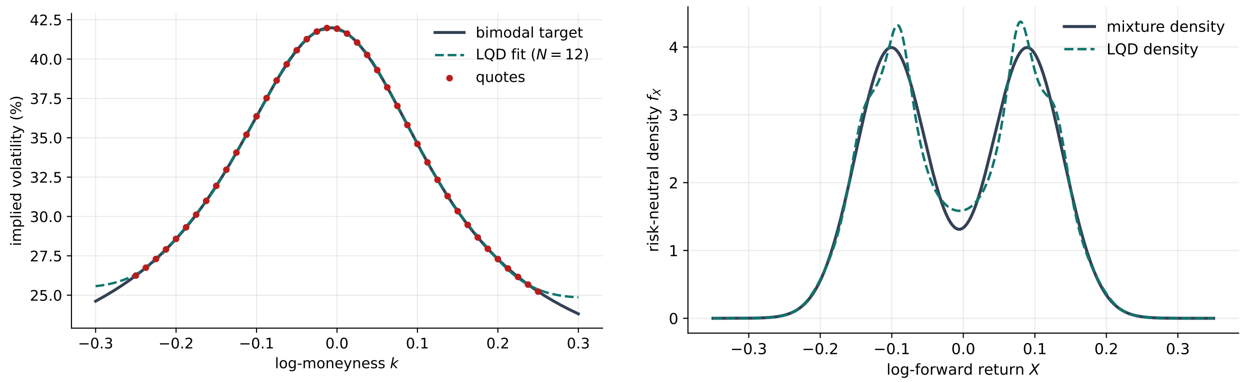

Figure 3: The bimodal event example. Left: target smile and $N = 12$ LQD fit — note the central trough and supported wings that a quadratic-in-k model cannot make. Right: the true mixture density and the LQD density, both with two hats.

11 What is genuinely original here

Quantile-based and density-based smile constructions are not new, and the Lee analysis is classical. Three things in this implementation are worth singling out:

1. **The endpoint basis $-\log u - \log(1-u) + (1-u)L + uR$.** It is what makes the tails *exponential with controllable scale* while keeping the body in a clean orthogonal-polynomial space; the Lee slopes (23) then come out of the parametrization rather than being patched on.

2. **The exact ATM handles and the orthogonal chart** (section 6). The level/skew/curvature are closed-form functionals of the quantile, so the model carries genuine trader coordinates *and* a chart in which the optimizer's shape modes do not disturb them — the same three handles that the graph extrapolator propagates in Note 14.

3. **The implicit-root cancellation** (theorem 1). The strike root's dependence on θ vanishing from the price gradient is what turns a $P + 1$-rebuild finite-difference Jacobian into a one-pass analytic one; the benchmark reaches the same fitted solution within the stated tolerance.

4. **The logistic admissibility chart** (section 7.2). Reparametrizing the right tail through $A_R = \operatorname{expit}(\rho)$ maps the integrability wall to infinity, so the calibrator becomes a genuinely unconstrained least-squares problem over *exactly* the admissible slices — with trust-region steps automatically compressed where the wall used to be.

12 Limitations

LQD removes butterfly arbitrage by construction but *not* calendar arbitrage across independently fitted expiries — and the cross-expiry machinery of section 9 is *soft*, finite-weight and optional, so a coupled surface is calendar-*controlled*, not calendar-guaranteed. Very high Legendre orders can fit noise and sculpt spurious density wiggles; this is a model-risk, not an arbitrage, problem, controlled by the ridge, bid–ask weighting and event priors. Several further limits deserve explicit statement:

- **Near the integrability wall.** As $A_R \to 1$ the forward integral degenerates: the tail corrections dominate, conditioning of the fit and of the retarget Newton solve deteriorates, and the barrier is doing real work. Fits that live near the wall should be treated as fragile even when admissible.
- **Atoms of any kind.** The model is a continuous law on $(0, \infty)$: not only a jump-to-default atom at zero but *any* exact mass point (e.g. a pinned settlement price) can only be approximated by a sharp continuous feature, never represented.
- **Positive is not plausible.** Structural positivity guarantees a *density*, not a sensible one: an unregularized high-order fit can produce economically implausible (multi-modal, spiky) yet perfectly arbitrage-free shapes. Positivity removes one failure class; judgment and the ridge handle the rest.
- **Coefficient non-uniqueness in practice.** Distinct coefficient vectors can price indistinguishably (near-degenerate directions grow with N and with sparse data), so fitted coefficients are not comparable across fits — compare handles, curves and densities, not raw θ.
- **Unstudied sensitivities.** There is no dedicated study of handle *drift* under rolling recalibration, nor of digital/one-touch sensitivities to the interpolation scheme; both are open measurement items, not established properties.

13 Traceability

One caveat on the "match note" anchors: the fixture coefficients those tests lock were recorded when the benchmark was first frozen, so they are *legacy locks* — they pin the production behaviour against regression, but their literal values can lag the freshly regenerated tables in this note (the note's numbers always come from a fresh `gen_lqd.py` run). Where the two disagree, the fresh run is the current truth and the test is the regression tripwire.

Table 4: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor *Test anchors*

Density positive; call curve monotone and convex; parity holds	proposition 1	`models/lqd/quadrature.py`
		`test_lqd_pricing.py::test_density_positive`
		`::test_call_curve_monotone_and_convex_in_strike`
		`::test_put_call_parity`

Claim	Object	Code anchor *Test anchors*

Martingale shift correct; $\mathbb{E}[e^X] = 1$	(11)	`models/lqd/quadrature.py`
		`test_lqd_pricing.py::test_svi_fit_martingale_shift_matches_note`
		`::test_svi_fit_martingale_integral_is_one`

Endpoint scales and Lee slopes; stable for extreme scales	(17)–(23)	`models/lqd/basis.py`
		`test_lqd_basis.py::test_svi_fit_endpoint_scales_match_note`
		`::test_svi_fit_lee_slopes_match_note`
		`::test_lee_slopes_stable_for_tiny_nonzero_endpoint_scales`

Legendre basis: recursion and orthogonality	(16)	`models/lqd/basis.py`
		`test_lqd_basis.py::test_legendre_recursion_matches_explicit_polynomials`
		`::test_legendre_orthogonality_on_unit_interval`

Exact ATM handles	(30)	`models/lqd/atm.py`
		`test_lqd_atm.py::test_atm_level_matches_implied_vol`
		`::test_atm_skew_matches_finite_difference`
		`::test_atm_curvature_matches_finite_difference`

Orthogonal chart: shape moves preserve handles; retarget exact	section 6.2	`models/lqd/ortho.py`
		`test_lqd_ortho.py::test_shape_move_leaves_handles_nearly_unchanged`
		`::test_retarget_hits_exact_handles`

Package controls move one package with handles fixed; GN metric prices moves by fit impact	section 6.3	`models/lqd/packages.py`
		`test_lqd_r5_controls.py::test_package_controls_move_their_package_and_not_the_handles`
		`::test_gauss_newton_chart_is_a_right_inverse_with_less_fit_impact`

Optimization charts: body modes endpoint-neutral; logistic cannot reach the wall; same optimum in every chart	section 7.2	`models/lqd/charts.py`
		`test_lqd_endpoint_coords.py::test_body_modes_are_endpoint_neutral`
		`::test_logistic_chart_cannot_reach_the_wall`
		`::test_logistic_fit_reaches_the_lr_optimum`

Analytic Jacobian matches FD (mid, band, calendar); fits agree	theorem 1	`models/lqd/jacobian.py`
		`test_lqd_jacobian.py::test_jacobian_matches_fd_mid`
		`::test_jacobian_matches_fd_band`
		`::test_jacobian_matches_fd_calendar`
		`::test_analytic_and_fd_fits_agree`

Var-swap rows on the analytic path; var-swap fit agrees with FD	section 7.5	`models/lqd/jacobian.py`
		`test_lqd_jacobian.py::test_jacobian_matches_fd_varswap_rows`
		`::test_varswap_fit_ungated_and_agrees_with_fd`

Claim	Object	Code anchor *Test anchors*

Calendar floor confined to price space on common support; symmetric repair leaves clean ladders untouched; desk-unit violation print	section 9	`calib/symmetric.py`
		`test_calendar_confinement.py`
		`test_symmetric_surface.py`
		`test_lqd_r5_controls.py::test_calendar_argmax_names_the_cheapest_trade`

Barrier finite for wild trial parameters	(34)	`models/lqd/calibrate.py` `test_lqd_jacobian.py::test_barrier_residual_finite_for_wild_tail_scale`

Benchmark fit and speed rail (legacy fixture)	section 10	`models/lqd/calibrate.py`
		`test_lqd_calibrate.py::test_calibrate_to_svi_benchmark`
		`::test_calibration_is_fast_enough`

Double-hat event smile: bimodal density recovered	section 10	`models/lqd/calibrate.py` `test_lqd_pricing.py::test_double_hat_density_is_bimodal`
JW conversion used by the benchmark	appendix C	`models/svi_jw/svi.py` `test_lqd_pricing.py::test_jw_conversion_matches_note`

A LQD-native controls and numerical constants

The knobs *specific to* the LQD slice, with defaults and roles. Surfaced knobs are user-facing `FitSettings`; hidden knobs are internal constants tuned to the algorithm and not exposed. The *shared* calibration controls that also shape an LQD fit — `fitMode`, `haircut`, `midAnchorWeight`, `enforceCalendar`/`calendarWeight`, the var-swap controls, and the prior-persistence and observation-filter settings of table 1 — are indexed in Note 00's control table and derived in their own notes (07, 08, 10, 13, 15); they are deliberately not duplicated here.

Table 5: LQD-native controls.

Knob	Default	Role
Surfaced (FitSettings)		
`nOrder` N	6	Legendre expansion order; 7 parameters at $N = 6$. Schema range $4 \le N \le 16$: 6 for ordinary slices, 8–14 for scheduled-event slices.
`lqdCoords`	`logistic`	Optimization chart (section 7.2): `lr` raw θ; `endpoint` $(\log A_L, \log A_R, a)$; `logistic` pushes A_R through $\mathrm{expit}(\rho)$ so the wall is unreachable. Optimum chart-independent to solver tolerance.
`regLambda` λ	10^{-6}	High-order ridge coefficient in (33); suppresses oscillation in sparse data.

Knob	Default	Role
`regPower` r	1.0	Ridge exponent in $n^{2r}a_n^2$; larger r penalizes high modes harder.
`barrierCenter` c	0.90	Centre of the A_R softplus barrier (34); engages before the $A_R = 1$ wall.
`barrierScale` s	50.0	Steepness of that barrier.
`weightScheme`	`equal`	Per-quote weights ω_i (`equal` or `tv_density`); see Note 07.
Hidden (internal constants)		
`Z_MAX` Z	40.0	Half-width of the symmetric logit quadrature grid; truncation error $\ll$ vega noise for equity tails.
`N_POINTS`	8001	Reporting/diagnostic grid nodes (odd, so $z = 0$ is a node).
`OPT_N_POINTS`	2001	Coarser grid used during the optimization iterations; accepted slice rebuilt at `N_POINTS`.
`EPS_AR` ε_R	10^{-6}	Buffer in the hard bound $A_R < 1 - \varepsilon_R$.
`_VEGA_FLOOR` η	10^{-4}	Floor added to Black vega in the residual normalization (25).
`xtol/ftol/gtol`	10^{-10}	`trf` tolerances; ~ 6 orders below the fit budget, so the optimum is display-identical while `trf` stops grinding invisible reductions.
`max_nfev`	4000	Evaluation cap (rarely approached).

B Performance notes

1. **Cached static grid.** The grid $(z, u, u(1-u))$ and the Legendre matrix depend only on (Z, n_{pts}, N), so they are LRU-cached and returned read-only; only $g(u)$ is re-formed per call. This removed $\sim 40\%$ of `build_slice` and made each build $\sim 1.7\times$ faster.

2. **Two-grid optimization.** Iterations run on `OPT_N_POINTS` $= 2001$ (Simpson error already far below the fit budget); the accepted slice is rebuilt once at 8001 nodes for the reported result, density, var-swap and calendar diagnostics. Converged parameters agree with a full-grid fit to $\sim 10^{-6}$.

3. **Hermite interpolation/inversion.** Storing the exact nodal derivatives $\mathrm{d}Q/\mathrm{d}z$ and $\mathrm{d}A/\mathrm{d}z$ gives cubic-Hermite off-grid evaluation (clean Greeks) and machine-precision strike inversion by Hermite–Newton — and supplies precisely the nodal sensitivities the analytic Jacobian reuses.

4. **Analytic Jacobian** (section 8). One quadrature pass instead of $P + 1$ finite-difference rebuilds, reaching the same fitted solution within tolerance (relative cost agreement $\sim 10^{-11}$), measured 1.3–1.7$\times$ on the residual-path benchmark of table 2 and structurally more on heavier configurations. Gated only on the prior-anchor and operator blocks; the var-swap gate was lifted in committee revision R5.

5. **Looser tolerances.** Dropping `trf` tolerances from 10^{-15} to 10^{-10} stops the optimizer chasing

a 10^{-15} cost reduction that is invisible in the priced surface, saving whole $(P+1)$-eval iterations at no display-precision cost.

C The SVI-JW conversion used in the example

SVI ("stochastic volatility inspired") is the five-parameter total-variance slice $w(k) = a + b\{\rho(k - m) + \sqrt{(k-m)^2 + \sigma_{\mathrm{SVI}}^2}\}$; SVI-JW ("jump-wings") re-coordinatizes it into trader quantities. The JW parameters $(v, \psi, p, c, \widetilde{v})$ relate to raw SVI by $w_0 = vT$, $b = \frac{\sqrt{w_0}}{2}(p+c)$, $\rho = \frac{c-p}{c+p}$, $\psi = \frac{b}{2\sqrt{w_0}}(\rho - \frac{m}{\sqrt{m^2+\sigma_{\mathrm{SVI}}^2}})$ and $\widetilde{v}T = a + b\sigma_{\mathrm{SVI}}\sqrt{1-\rho^2}$. Setting $\chi = \frac{m}{\sqrt{m^2+\sigma_{\mathrm{SVI}}^2}} = \rho - \frac{4\psi}{p+c}$ gives $m = \chi\sigma_{\mathrm{SVI}}/\sqrt{1-\chi^2}$ and $\sqrt{m^2+\sigma_{\mathrm{SVI}}^2} = \sigma_{\mathrm{SVI}}/\sqrt{1-\chi^2}$, and subtracting the minimum-variance relation from the ATM relation,

$$\sigma_{\mathrm{SVI}} = \frac{w_0 - \widetilde{v}T}{b((1-\rho\chi)/\sqrt{1-\chi^2} - \sqrt{1-\rho^2})}, \quad m = \frac{\chi\sigma_{\mathrm{SVI}}}{\sqrt{1-\chi^2}}, \quad a = \widetilde{v}T - b\sigma_{\mathrm{SVI}}\sqrt{1-\rho^2}. \tag{45}$$

The inversion is exact on its documented domain — $p + c > 0$, $|\chi| < 1$ and a nonzero denominator above — and has a genuine *coordinate singularity* at $\psi = 0$ with $m \neq 0$ handled separately (Note 02 states the full domain contract, $-p/2 < \psi < c/2$, and its $\psi = 0$ stratum). The benchmark's raw parameters in section 10 are the image, at $T = 0.5$, of

$$(v, \psi, p, c, \widetilde{v}) = (0.0425,\ -0.25,\ 0.75,\ 0.25,\ 0.034).$$

D Reference implementation

The whole slice is one quadrature pass in the logit coordinate $z = \log(u/(1-u))$: form g, integrate $\mathrm{d}Q/\mathrm{d}z = e^g$ to the quantile, solve one scalar normalization for the martingale shift μ, and reverse-integrate the asset-share $A(z)$; every strike is then priced by $C(k) = A(z_k) - e^k(1-u_k)$. The listing below reproduces `build_slice`'s quantile, asset share and μ to 10^{-8} on the $N = 6$ benchmark and gives $\mathbb{E}[e^X] = 1.000000$ (production inverts $Q(z_k) = k$ and interpolates A with cubic Hermite for smooth Greeks; the linear `np.interp` here agrees to 10^{-7} on the dense grid).

Listing 2: The LQD quadrature pipeline (distilled from `models/lqd/{basis,quadrature}.py`).

```python
def legendre(n_max, x):                 # P_0..P_{n_max}, 3-term recursion
    P = np.empty((n_max+1, x.size)); P[0] = 1.0; P[1] = x
    for n in range(1, n_max):
        P[n+1] = ((2*n+1)*x*P[n] - n*P[n-1]) / (n+1)
    return P

def g_of_u(u, L, R, a):                 # smooth part g(u) of ell(u)
    return (1-u)*L + u*R + a @ legendre(len(a)+1, 1 - 2*u)[2:]

def build_slice(z, L, R, a):
    # One quadrature pass, eqs (q_logit)-(mu_norm).
    u = expit(z); dz = z[1] - z[0]
    # Tail scales e^{g(0)}, e^{g(1)}:
    A_L, A_R = np.exp(g_of_u(np.array([0.0, 1.0]), L, R, a))
```

```
    dQ = np.exp(g_of_u(u, L, R, a))          # dQ/dz = e^{g}
    Q = cumulative_simpson(dQ, dx=dz, initial=0.0)
    Q -= Q[len(z)//2]                        # anchor Q(0) = 0
    mass = np.exp(Q)*u*(1-u)
    tot = (np.trapezoid(mass, z)             # + analytic tails
           + np.exp(Q[-1]-z[-1])/(1-A_R)
           + np.exp(Q[0]-z[-1])/(1+A_L))
    mu = -np.log(tot); Q = Q + mu            # now E[e^X] = 1
    m = np.exp(Q)*u*(1-u)
    # Asset share A(z) = int_z^inf e^{Q} u(1-u):
    A = (cumulative_simpson(m[::-1], dx=dz, initial=0.0)[::-1]
         + np.exp(Q[-1]-z[-1])/(1-A_R))
    return u, Q, A

def call_price(k, z, u, Q, A):
    # C(k) = A(z_k) - e^k (1 - u_k)
    zk = np.interp(k, Q, z)                  # invert Q(z_k) = k
    return np.interp(zk, z, A) - np.exp(k)*(1 - expit(zk))
```

References

[1] D. Breeden and R. Litzenberger. Prices of state-contingent claims implicit in option prices. *Journal of Business*, 51(4):621–651, 1978.

[2] R. W. Lee. The moment formula for implied volatility at extreme strikes. *Mathematical Finance*, 14(3):469–480, 2004.

[3] J. Gatheral. *The Volatility Surface: A Practitioner's Guide*. Wiley, 2006.

[4] J. Gatheral and A. Jacquier. Arbitrage-free SVI volatility surfaces. *Quantitative Finance*, 14(1):59–71, 2014.

[5] V. Strassen. The existence of probability measures with given marginals. *Annals of Mathematical Statistics*, 36(2):423–439, 1965.

[6] P. Carr and D. Madan. Towards a theory of volatility trading. In *Volatility*, Risk Books, 1998.